`$e->getMessage()` simply means "call the `getMessage()` method of the `$e` object". `getMessage()` then accesses some internal string data of `$e` and returns it: you'll notice it's the same string we used to construct the `Exception` in the `boom()` function. Crucially, we don't trigger a fatal error, because this time we provided PHP with an alternative logical path in our code, and execution continues to the final line.

PHP 7 Errors

The core `Exception` class is not the only game in town. Version 7 upped its game with exceptions: many extensions switched from emitting errors to throwing exceptions, and many core error conditions were given their own exceptions to throw, all of which are of the `Error` class (or classes derived from `Error`, which we'll get to shortly). Here's an example:

```
try {
    notDefined();
} catch (Error $e) {
    print "Caught an Error" . PHP_EOL;
    print $e->getMessage() . PHP_EOL;
}

print "Carrying on..." . PHP_EOL;
notDefined();

Output:
Caught an Error
Call to undefined function notDefined()
Carrying on...
PHP Fatal error:  Uncaught Error: Call to undefined function notDefined()
```

Attempting to call an undefined function in PHP 7 onwards results in an instance of the `Error` class being thrown. `Error` objects behave just like `Exception` objects, and the `throw` language construct treats them identically, as does `catch` et al. Prior to version 7, calling an undefined function resulted in an `E_ERROR` emission. Here, in PHP 8, the second time we try to call `notDefined()`, we also end up with an `E_ERROR` emitted, but that's because the `Error` exception wasn't caught; the illegal operation itself did not emit the error. A subtle but important difference: The `PHP Fatal error: Uncaught...` message means you had a chance to handle the problem!